UNIT IV: Inheritance & Pointers

Question Bank Answers: Debugging & Pseudo-Code Formulation

Name	Nikitha R
Reg. No	19252421
Course Code & Name	DSA0105 Object Oriented Programming with C++

Contents

- **Section A: Debugging** (5 problems): line numbers and defect category, theory (class layouts, pointer type checking), corrected code and dry run with console output.
- **Section B: Pseudo-Code** (5 problems): design overview, structured pseudo-code (EXTENDS / VIRTUAL / BASE(...) notation) and a trace table.
- Every corrected C++ program was compiled with g++ (`-Wall -Wextra`) and run; the outputs shown are the real outputs. The buggy originals were compiled too, so the quoted compiler messages are real GCC messages.

SECTION A: DEBUGGING QUESTIONS

Question 1: The Diamond Problem in Multipath Hybrid Inheritance

Topic: Hybrid Inheritance, Virtual Base Classes, Ambiguity

Step 1: Analyse the problem statement

- **Expected:** a TeachingAssistant is one person with one ID.
- **Actual:** because `Student` and `Employee` each inherit `Person` normally, every TeachingAssistant contains *two* Person parts.

Step 2: Error found (Task 1)

Line	Classification	Defect
Line 33: `ta.idNumber`	Compilation Error (ambiguity)	GCC: **"request for member 'idNumber' is ambiguous"**. Root cause is on Lines 12 and 18: `Student` and `Employee` inherit `Person` without `virtual`.

Step 3: Root cause explanation (Tasks 2 to 4)

Memory layout under non-virtual inheritance (each base class becomes an embedded subobject of the derived object):

```
TeachingAssistant object
+--------------------------------+
| Student subobject              |
|   +- Person subobject          |
|        idNumber  (= 101)       |   <- Student::Person::idNumber
+--------------------------------+
| Employee subobject             |
|   +- Person subobject          |
|        idNumber  (= 202)       |   <- Employee::Person::idNumber
+--------------------------------+
```

- **Why two idNumber fields.** A non-virtual base is copied into *every* path that inherits it. There are two inheritance paths from `TeachingAssistant` to `Person` (through Student and through Employee), so two complete Person subobjects exist, with two constructor calls (the buggy program prints "Person initialized: 101" then "Person initialized: 202"). The name `idNumber` finds two candidates at equal depth, so lookup is ambiguous. (Explicit disambiguation `ta.Student::idNumber` compiles, but the object still holds two inconsistent IDs, which is a design error.)
- **Fix: virtual base class.** Declaring `class Student : virtual public Person` and `class Employee : virtual public Person` makes all paths share **one** Person subobject, located through a hidden virtual-base pointer/offset stored in each intermediate class. `ta.idNumber` is then unambiguous.
- **Constructor delegation changes.** Normal rule: a class initializes only its direct bases. For a virtual base, the **most derived class** (here `TeachingAssistant`) is solely responsible for constructing it, and it does so *first*, before any non-virtual base. Therefore `TeachingAssistant` must call `Person(sId)` in its own initializer list. The `Person(id)` calls written in the Student and Employee constructors are *ignored* when those classes are built as part of a TeachingAssistant (they only run when a Student or Employee is created on its own). If TeachingAssistant forgets, `Person(int id = 0)` is used with id 0, silently.

Step 4: Corrected code

```
#include <iostream>
#include <string>

class Person {
public:
    int idNumber;
    Person(int id = 0) : idNumber(id) {
        std::cout << "Person initialized: " << idNumber << std::endl;
    }
};

// 'virtual' makes Student and Employee share ONE Person subobject
class Student : virtual public Person {
public:
    Student(int id) : Person(id) {}
};

class Employee : virtual public Person {
public:
    Employee(int id) : Person(id) {}
};

class TeachingAssistant : public Student, public Employee {
public:
    // Most derived class initializes the virtual base directly
    TeachingAssistant(int sId, int eId)
        : Person(sId), Student(sId), Employee(eId) {}
};

int main() {
    TeachingAssistant ta(101, 202);

    std::cout << "TA ID: " << ta.idNumber << std::endl;   // unambiguous now
    return 0;
}
```

Step 5: Dry run and output

Order	Constructor	Reason
1	Person(101)	virtual base is constructed first, by the most derived class's initializer
2	Student(101)	non-virtual base in declaration order; its Person(id) call is skipped
3	Employee(202)	next base; its Person(id) call is skipped
4	TeachingAssistant body	runs last

```
Person initialized: 101
TA ID: 101
```

Person is initialized exactly once, with the value passed by TeachingAssistant (101); the 202 passed to Employee does not reach Person.

Question 2: Missing Base Constructor in Derived Initializer List

Topic: Constructors in Derived Classes, Order of Constructor Execution

Step 1: Analyse the problem statement

- **Expected:** `Sensor` passes its MAC address up to `Device`.
- **Actual:** the code does not compile.

Step 2: Error found

Line	Classification	Defect
Line 18: `Sensor(std::string mac, int rate) { ... }`	Compilation Error	GCC: **"no matching function for call to 'Device::Device()'"**. No initializer names `Device`, and `Device` has no default constructor.

Step 3: Root cause explanation (Tasks 1 and 2)

- **Why the error appears before the body.** A constructor body executes only *after* every base subobject and data member has been initialized. If the member initializer list does not mention a base class, the compiler default-initializes it, i.e. it implicitly calls the base's **zero-argument constructor** `Device::Device()` as the first step of `Sensor`'s constructor. `Device` declares only `Device(std::string)`, which suppresses the implicit default constructor, so that call has no target. The assignment `macAddress = mac;` in the body is too late (and would only be an assignment, not initialization).
- **Chronological order of construction for a derived object:** (1) virtual base classes, in depth-first left-to-right order of declaration; (2) direct non-virtual base classes, in the order they appear in the class's *base list* (not the initializer-list order); (3) data members (including member objects), in the order they are *declared* in the class (not initializer-list order); (4) the body of the derived constructor. Destruction runs in exactly the reverse order.
- **Fix.** Put the base constructor call in the member initializer list: `Sensor(std::string mac, int rate) : Device(mac), samplingRate(rate) { ... }`.

Step 4: Corrected code

```
#include <iostream>
#include <string>

class Device {
protected:
    std::string macAddress;

public:
    Device(std::string mac) : macAddress(mac) {
        std::cout << "Device configured with MAC: " << macAddress << std::endl;
    }
};

class Sensor : public Device {
private:
    int samplingRate;

public:
    // Member initializer list: Device(mac) runs BEFORE the Sensor body
    Sensor(std::string mac, int rate) : Device(mac), samplingRate(rate) {
        std::cout << "Sensor configured at " << samplingRate << " Hz" << std::endl;
    }

    void display() const {
        std::cout << "MAC: " << macAddress << ", Rate: " << samplingRate << " Hz" << std::endl;
    }
};

int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

Step 5: Dry run and verified output

Step	Action	Output
1	Sensor("00:1A:2B:3C:4D:5E", 100) starts: initializer list calls Device(mac) first	Device configured with MAC: ...
2	samplingRate = 100 initialized (member)	-
3	body of Sensor runs	Sensor configured at 100 Hz
4	tempSensor.display()	MAC: ..., Rate: 100 Hz

```
Device configured with MAC: 00:1A:2B:3C:4D:5E
Sensor configured at 100 Hz
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz
```

Question 3: Base Class Pointer Member Access & Object Slicing

Topic: Pointers to Derived Classes, Static Type Checking, Object Slicing

Step 1: Analyse the problem statement

- **Expected:** use a `Vehicle*` that points to a `SportsCar` and still reach turbo features.
- **Actual:** a compile error at Line 22, and silent data loss at Line 26. A third, quieter defect also exists (see Step 2).

Step 2: Errors found

Line	Classification	Defect
Line 22: `vPtr->engageTurbo()`	Compilation Error	GCC: "'class Vehicle' has no member named 'engageTurbo'".
Line 26: `Vehicle plainVehicle = car;`	Logic Defect (Object Slicing)	Compiles; the SportsCar part is cut off.
Line 29: `delete vPtr;`	Undefined Behavior (additional)	Deleting a derived object through a base pointer when `Vehicle` has a non-virtual destructor (GCC with -Wall warns "-Wdelete-non-virtual-dtor" where applicable).

Step 3: Root cause explanation (Tasks 1 to 3)

- **Static type checking (Line 22).** Member lookup for `p->name` is done at *compile time* using the **declared type of the pointer**, not the type of the object it will point to at run time. `vPtr` is declared `Vehicle*`, so only members of `Vehicle` (and its bases) are visible. The compiler cannot know, and does not try to prove, that the object is a `SportsCar`. The heap object does contain `turboEnabled` and `engageTurbo`, but they are invisible through this pointer type.
- **Object slicing (Line 26).** `Vehicle plainVehicle` is a complete, separate object of type `Vehicle`, sized for Vehicle's members only. The initialization uses Vehicle's copy constructor, which accepts the `SportsCar` as a `const Vehicle&` and copies just the Vehicle subobject. `turboEnabled` **is discarded** (it has nowhere to live), the object's dynamic type becomes `Vehicle`, and for polymorphic classes the vptr is the Vehicle's, so overridden behavior is lost too.
- **Correct access.** Use an explicit downcast when the real type is known: `static_cast<SportsCar*>(vPtr)->engageTurbo();` (use `dynamic_cast` with a polymorphic base when the type must be checked at run time).
- **Preventing slicing.** Bind a *reference* or *pointer* to the original object (`Vehicle &ref = car;`, `Vehicle *p = &car;`) and pass objects to functions by reference/pointer, never by value. A reference is just another name for the same full-size object, so nothing is cut off.
- **Destructor.** Make `~Vehicle()` virtual so that `delete vPtr;` runs the correct destructor chain.

Step 4: Corrected code

```
#include <iostream>

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() {}                       // needed: deleting a SportsCar through Vehicle*
    void honk() { std::cout << "General Horn" << std::endl; }
};

class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
    void engageTurbo() { std::cout << "Turbo Engaged!" << std::endl; }
};

int main() {
    Vehicle *vPtr = new SportsCar(300, true);   // upcast

    vPtr->honk();

    // Explicit downcast: the programmer asserts the real type
    static_cast<SportsCar*>(vPtr)->engageTurbo();

    SportsCar car(250, true);

    Vehicle sliced = car;                       // slicing: turboEnabled is lost
    std::cout << "Sliced copy maxSpeed: " << sliced.maxSpeed << std::endl;

    Vehicle &ref = car;                         // no slicing: same object
    std::cout << "Via reference maxSpeed: " << ref.maxSpeed << std::endl;
    static_cast<SportsCar&>(ref).engageTurbo(); // full SportsCar still reachable

    delete vPtr;
    return 0;
}
```

Step 5: Dry run and output

Statement	Meaning	Output
vPtr->honk()	found in Vehicle	General Horn
static_cast<SportsCar*>(vPtr)->engageTurbo()	downcast to the true type	Turbo Engaged!
Vehicle sliced = car	copies maxSpeed only; turboEnabled lost	Sliced copy maxSpeed: 250
Vehicle &ref = car	alias of the full object	Via reference maxSpeed: 250
static_cast<SportsCar&>(ref).engageTurbo()	full SportsCar still reachable	Turbo Engaged!
delete vPtr	virtual destructor, whole SportsCar released	(no output)

```
General Horn
Turbo Engaged!
Sliced copy maxSpeed: 250
Via reference maxSpeed: 250
Turbo Engaged!
```

Question 4: Incorrect Return Type in this Pointer Method Chaining

Topic: The this Pointer, Method Chaining (Fluent Interface)

Step 1: Analyse the problem statement

- **Expected:** `box.setWidth(800).setHeight(600);` configures both dimensions.
- **Actual:** the chain fails to compile.

Step 2: Error found

Line	Classification	Defect
Line 11: `WindowBox* setWidth(int width)` returning `this`	Compilation Error (at Line 30)	GCC: "request for member 'setHeight' in 'box.WindowBox::setWidth(800)', which is of pointer type 'WindowBox*' (maybe you meant to use '->' ?)".

Step 3: Root cause explanation (Tasks 1 and 2)

- **Line 30.** `box.setWidth(800)` evaluates to a **pointer** (`WindowBox*`). The dot operator selects members of an *object* or reference; a pointer needs the arrow `->`. So `.setHeight` applied to a pointer is a syntax/type error. `box.setWidth(800)->setHeight(600)` would compile but breaks the uniform fluent style, and mixes two return conventions in one class.
- `this` **versus** `*this`**.** Inside a non-static member function, `this` is a hidden pointer parameter of type `WindowBox*` (`const WindowBox*` in a const member): a value holding the *address* of the object the function was called on. Dereferencing it, `*this`, yields the *object itself* as an lvalue of type `WindowBox`. Returning `*this` from a function whose return type is `WindowBox&` hands back an alias of the same object (no copy), so the next call in the chain acts on the same `box`.
- **Danger to avoid.** Returning by value (`WindowBox` with `return *this;`) would copy the object, so the second call would modify the copy and the original would keep only the first change.
- **Fix.** Change Line 11 to `WindowBox& setWidth(int width)` and `return *this;`.

Step 4: Corrected code

```
#include <iostream>

class WindowBox {
private:
    int width;
    int height;

public:
    WindowBox() : width(0), height(0) {}

    WindowBox& setWidth(int width) {            // returns a reference
        this->width = width;
        return *this;
    }

    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
    }

    void display() const {
        std::cout << "Window: " << width << " x " << height << std::endl;
    }
};

int main() {
    WindowBox box;

    box.setWidth(800).setHeight(600);           // chaining works

    box.display();
    return 0;
}
```

Step 5: Dry run and output

Step	Expression	Result
1	box.setWidth(800)	width = 800; returns reference to box
2	.setHeight(600) on that reference	height = 600; returns reference to box
3	box.display()	Window: 800 x 600

```
Window: 800 x 600
```

Question 5: Private Inheritance Access Violations & Inaccessible Base Cast

Topic: Access Specifiers in Inheritance (private derivation), Upcasting

Step 1: Analyse the problem statement

- **Expected:** `SecureChannel` reuses `RawSocket`'s implementation without being a public kind of RawSocket.
- **Actual:** two statements in `main` are rejected.

Step 2: Errors found

Line	Classification	Defect (GCC)
Line 24: `channel.closePort()`	Compilation Error (access)	"'void RawSocket::closePort()' is inaccessible within this context".
Line 27: `RawSocket *socketPtr = &channel;`	Compilation Error (inaccessible base)	"'RawSocket' is an inaccessible base of 'SecureChannel'".

Step 3: Root cause explanation (Tasks 1 to 3)

Inheritance	Base public members become	Base protected members become	Upcast Derived* to Base*
public	public	protected	allowed everywhere
protected	protected	protected	only in members/friends of Derived and its descendants
private	private	private	only in members/friends of Derived

- **Task 1.** With `private` derivation, `openPort()` and `closePort()` (public in RawSocket) become **private members of SecureChannel**. Member functions of SecureChannel can use them (so `establishEncryptedTunnel` may call `openPort`), but code outside the class cannot. `main()` is outside, so Line 24 is an access violation. Private inheritance models "implemented in terms of", not "is-a".
- **Task 2.** An implicit conversion from `Derived*` to `Base*` is permitted only when the base class is *accessible* at the point of conversion. Here the base relationship itself is private, so outside SecureChannel the conversion is ill-formed ("inaccessible base"). Allowing it would let any client use a `RawSocket*` to bypass the encapsulation that private inheritance was chosen to provide. Upcasting is allowed (a) always for public inheritance, (b) inside members/friends of the derived class for private inheritance, (c) inside derived classes for protected inheritance.
- **Task 3, Approach A.** Add the using-declaration `using RawSocket::closePort;` in the *public* section of `SecureChannel`: only that one name is re-exposed; `openPort` stays hidden, and the base stays private (the upcast on Line 27 remains illegal, which is desirable).
- **Task 3, Approach B.** Either use **public inheritance** (changes the meaning to "is-a": every SecureChannel can then be used as a RawSocket, including through `RawSocket*`), or, better when the relationship is "has-a", use **composition**: keep a private `RawSocket socket;` member and write forwarding methods `void closePort() { socket.closePort(); }`. Composition gives the same reuse without any inheritance conversions.

Step 4: Corrected code using Approach A

```
#include <iostream>

class RawSocket {
public:
    void openPort(int port) {
        std::cout << "Port " << port << " opened." << std::endl;
    }
    void closePort() {
        std::cout << "Port closed." << std::endl;
    }
};

class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;                 // Approach A: re-expose one member as public

    void establishEncryptedTunnel(int port) {
        openPort(port);
        std::cout << "Tunnel encrypted with TLS 1.3" << std::endl;
    }
};

int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);

    channel.closePort();                        // legal now

    // RawSocket *socketPtr = &channel;         // still illegal outside: base is private
    return 0;
}
```

Step 5: Dry run and output

Statement	Check	Result
channel.establishEncryptedTunnel(443)	public member of SecureChannel; calls private-inherited openPort inside the class	Port 443 opened. / Tunnel encrypted with TLS 1.3
channel.closePort()	accessible through the using-declaration	Port closed.
RawSocket *socketPtr = &channel	removed (commented out): illegal outside the class	-

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Topic: Hybrid Inheritance, Virtual Base Classes, Diamond Resolution

Design overview

- **Diamond:** Person is the root; Student and Faculty inherit it with VIRTUAL, so a TeachingAssistant contains one Person subobject.
- **Constructor rule:** the most derived class (TeachingAssistant) initializes the virtual base Person directly: `CONSTRUCTOR(...) : Person(...), Student(...), Faculty(...)`.
- **Unambiguous access:** personID, fullName (from the single Person), gpa (Student), academicDepartment (Faculty) and stipendAmount (own) are each found by exactly one path.

Structured pseudo-code

```
CLASS Person
  PROTECTED:
    STRING personID, fullName, email
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING name, STRING mail)
      personID <- id ; fullName <- name ; email <- mail
      DISPLAY "Person constructed"
    ENDCONSTRUCTOR
ENDCLASS

CLASS Student INHERITS VIRTUAL Person
  PROTECTED:
    REAL    gpa
    INTEGER creditsEarned
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING name, STRING mail, REAL g, INTEGER credits) : Person(id, name, mail)
      gpa <- g ; creditsEarned <- credits
      DISPLAY "Student constructed"
    ENDCONSTRUCTOR
ENDCLASS

CLASS Faculty INHERITS VIRTUAL Person
  PROTECTED:
    STRING academicDepartment, rankTitle
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING name, STRING mail, STRING dept, STRING rank) : Person(id, name, mail)
      academicDepartment <- dept ; rankTitle <- rank
      DISPLAY "Faculty constructed"
    ENDCONSTRUCTOR
ENDCLASS

CLASS TeachingAssistant INHERITS Student, Faculty
  PRIVATE:
    REAL   stipendAmount
    STRING assignedCourse
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING name, STRING mail, REAL g, INTEGER credits,
                STRING dept, STRING rank, REAL stipend, STRING course)
        : Person(id, name, mail),                  // virtual base: initialized HERE, first
          Student(id, name, mail, g, credits),     // its Person(...) call is ignored
          Faculty(id, name, mail, dept, rank)      // its Person(...) call is ignored
      stipendAmount  <- stipend
      assignedCourse <- course
      DISPLAY "TeachingAssistant constructed"
    ENDCONSTRUCTOR

    METHOD DisplayTARecord()
      DISPLAY "ID         : ", personID           // single Person subobject: no ambiguity
```

```
      DISPLAY "Name       : ", fullName
      DISPLAY "GPA        : ", gpa
      DISPLAY "Department : ", academicDepartment
      DISPLAY "Stipend    : ", stipendAmount
      DISPLAY "Course     : ", assignedCourse
    ENDMETHOD
ENDCLASS

BEGIN Main
  TeachingAssistant ta("P-1001", "Arun Kumar", "arun@univ.edu", 3.8, 90,
                       "Computer Science", "Teaching Fellow", 12000.0, "CS101")
  ta.DisplayTARecord()
END Main
```

Trace: instantiation with sample data and output

Order	Event	Printed
1	Person constructed by TeachingAssistant's initializer (virtual base first)	Person constructed
2	Student constructed (non-virtual base, declaration order; Person(...) skipped)	Student constructed
3	Faculty constructed (Person(...) skipped)	Faculty constructed
4	TeachingAssistant body: stipendAmount = 12000, assignedCourse = CS101	TeachingAssistant constructed
5	DisplayTARecord()	see below

```
ID         : P-1001
Name       : Arun Kumar
GPA        : 3.8
Department : Computer Science
Stipend    : 12000
Course     : CS101
```

Person is constructed exactly once (not twice), which is the whole point of the virtual base.

Question 2: Polymorphic Payroll with Abstract Base Class & Base Pointers

Topic: Abstract Classes, Pure Virtual Functions, Pointers to Derived Classes

Design overview

- **ABSTRACT CLASS Employee** declares ABSTRACT METHOD CalculatePay(), so it cannot be instantiated; each subclass must supply its own rule.
- **Base-pointer array:** ARRAY[2] OF POINTER TO Employee holds a SalariedEmployee and an HourlyEmployee; the call `staff[i]->CalculatePay()` selects the right override at run time (late binding).
- Employee has a VIRTUAL DESTRUCTOR so that deleting through an Employee pointer destroys the full derived object.
- Overtime rule: pay = wage × hours for hours <= 40; otherwise 40 × wage + (hours - 40) × wage × 1.5.

Structured pseudo-code

```
ABSTRACT CLASS Employee
  PROTECTED:
    STRING employeeID
    STRING name
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING empName)
      employeeID <- id
      name       <- empName
    ENDCONSTRUCTOR
    VIRTUAL DESTRUCTOR()
    ABSTRACT METHOD CalculatePay() RETURNS REAL        // pure virtual
    METHOD DisplayHeader()
      DISPLAY "Employee ", employeeID, " : ", name
    ENDMETHOD
ENDCLASS

CLASS SalariedEmployee EXTENDS Employee
  PRIVATE:
    REAL monthlySalary
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING n, REAL salary) : Employee(id, n)
      monthlySalary <- salary
    ENDCONSTRUCTOR
    OVERRIDE METHOD CalculatePay() RETURNS REAL
      RETURN monthlySalary
    ENDMETHOD
ENDCLASS

CLASS HourlyEmployee EXTENDS Employee
  PRIVATE:
    REAL hourlyWage
    REAL hoursWorked
  PUBLIC:
    CONSTRUCTOR(STRING id, STRING n, REAL wage, REAL hours) : Employee(id, n)
      hourlyWage  <- wage
      hoursWorked <- hours
    ENDCONSTRUCTOR
    OVERRIDE METHOD CalculatePay() RETURNS REAL
      REAL regularPay, overtimePay
      IF hoursWorked <= 40 THEN
        RETURN hoursWorked * hourlyWage
      ELSE
        regularPay  <- 40 * hourlyWage
        overtimePay <- (hoursWorked - 40) * hourlyWage * 1.5
        RETURN regularPay + overtimePay
      ENDIF
    ENDMETHOD
ENDCLASS
```

```
BEGIN Main
  DECLARE staff AS ARRAY[2] OF POINTER TO Employee
  REAL total, pay
  INTEGER i
  staff[0] <- NEW SalariedEmployee("E01", "Meera", 8500.0)
  staff[1] <- NEW HourlyEmployee("E02", "Ravi", 50.0, 48.0)
  total <- 0.0
  FOR i <- 0 TO 1 STEP 1
    staff[i]->DisplayHeader()
    pay   <- staff[i]->CalculatePay()               // dynamic dispatch
    total <- total + pay
    DISPLAY "Pay = ", pay
  ENDFOR
  DISPLAY "Total disbursement = ", total
  FOR i <- 0 TO 1 STEP 1
    DELETE staff[i]                                 // virtual destructor
  ENDFOR
END Main
```

Trace table

i	Object (dynamic type)	CalculatePay() path	Computation	pay	running total
start	-	-	-	-	0
0	SalariedEmployee	returns monthlySalary	8,500	8,500	8,500
1	HourlyEmployee (48 h, $50)	hoursWorked 48 > 40: overtime branch	regular = 40 × 50 = 2,000; overtime = (48 - 40) × 50 × 1.5 = 600	2,600	**11,100**

Question 3: Composite Canvas Engine with Member Classes & this Chaining

Topic: Member Classes (Composition), this Pointer Chaining

Design overview

- **Composition (has-a):** Circle contains a Point2D member centerPoint. The member object is built by Circle's constructor *before* the Circle body runs (member initializer list).
- **Fluent interface:** SetCenter and SetRadius return REF Circle via `RETURN REF THIS` (the object itself, not a copy), so calls chain left to right.
- SetRadius validates r > 0 and leaves the radius unchanged otherwise, but still returns REF THIS so the chain does not break.

Structured pseudo-code

```
CLASS Point2D
  PRIVATE:
    REAL x, y
  PUBLIC:
    CONSTRUCTOR(REAL px <- 0.0, REAL py <- 0.0)
      x <- px
      y <- py
    ENDCONSTRUCTOR
    METHOD Set(REAL px, REAL py)
      x <- px
      y <- py
    ENDMETHOD
    METHOD GetX() RETURNS REAL
      RETURN x
    ENDMETHOD
    METHOD GetY() RETURNS REAL
      RETURN y
    ENDMETHOD
ENDCLASS

CLASS Circle
  PRIVATE:
    Point2D centerPoint                          // member object (has-a)
    REAL    radius
  PUBLIC:
    CONSTRUCTOR() : centerPoint(0.0, 0.0)        // member built first
      radius <- 1.0
    ENDCONSTRUCTOR

    METHOD SetCenter(REAL newX, REAL newY) RETURNS REF Circle
      centerPoint.Set(newX, newY)
      RETURN REF THIS
    ENDMETHOD

    METHOD SetRadius(REAL r) RETURNS REF Circle
      IF r > 0 THEN
        radius <- r
      ELSE
        DISPLAY "Invalid radius ignored"
      ENDIF
      RETURN REF THIS
    ENDMETHOD

    METHOD Draw()
      DISPLAY "Circle: center = (", centerPoint.GetX(), ", ", centerPoint.GetY(), "), radius = ", radius
    ENDMETHOD
ENDCLASS
```

```
BEGIN Main
  Circle myCircle
  myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
END Main
```

Trace: myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()

Step	Construction / call	Receiver	centerPoint	radius	Returns
0	Circle constructed: Point2D(0,0) first, then body	myCircle	(0.0, 0.0)	1.0	-
1	SetCenter(12.5, 30.0)	myCircle	(12.5, 30.0)	1.0	REF myCircle
2	SetRadius(7.5): 7.5 > 0, accepted	the returned reference = myCircle	(12.5, 30.0)	7.5	REF myCircle
3	Draw()	the returned reference = myCircle	(12.5, 30.0)	7.5	prints the line below

```
Circle: center = (12.5, 30), radius = 7.5
```

All three calls act on the *same* object because each intermediate result is a reference to it; if SetCenter returned by value, SetRadius would change a temporary copy and myCircle would keep radius 1.0.

Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

Topic: Multilevel Inheritance, Constructors in Derived Classes, Base Pointers

Design overview

- **Chain:** Vehicle, then AutonomousVehicle EXTENDS Vehicle, then DeliveryDrone EXTENDS AutonomousVehicle. Each constructor passes the arguments its parent needs through the initializer list `: BASE(...)`.
- **Construction runs root to leaf** (Vehicle, AutonomousVehicle, DeliveryDrone); destruction runs leaf to root. Vehicle has a VIRTUAL DESTRUCTOR so the drone can be deleted through any base pointer.
- **Pointer management:** the drone is created on the heap; its address is stored in a `POINTER TO Vehicle` (upcast, implicit), which exposes only Vehicle members. A DOWNCAST to `POINTER TO DeliveryDrone` restores the full interface.

Structured pseudo-code

```
CLASS Vehicle
  PROTECTED:
    STRING vehicleVIN
    REAL   baseWeightKg
  PUBLIC:
    CONSTRUCTOR(STRING vin, REAL weight)
      vehicleVIN <- vin ; baseWeightKg <- weight
      DISPLAY "1. Vehicle constructed"
    ENDCONSTRUCTOR
    VIRTUAL DESTRUCTOR()
      DISPLAY "Vehicle destroyed"
    ENDDESTRUCTOR
    METHOD DisplaySpecs()
      DISPLAY "VIN: ", vehicleVIN, "  Base weight (kg): ", baseWeightKg
    ENDMETHOD
ENDCLASS

CLASS AutonomousVehicle EXTENDS Vehicle
  PROTECTED:
    STRING lidarSensorID
    STRING firmwareVersion
  PUBLIC:
    CONSTRUCTOR(STRING vin, REAL weight, STRING lidar, STRING fw) : Vehicle(vin, weight)
      lidarSensorID <- lidar ; firmwareVersion <- fw
      DISPLAY "2. AutonomousVehicle constructed"
    ENDCONSTRUCTOR
    DESTRUCTOR()
      DISPLAY "AutonomousVehicle destroyed"
    ENDDESTRUCTOR
    METHOD DisplayAutonomy()
      DISPLAY "LiDAR: ", lidarSensorID, "  Firmware: ", firmwareVersion
    ENDMETHOD
ENDCLASS

CLASS DeliveryDrone EXTENDS AutonomousVehicle
  PRIVATE:
    REAL payloadCapacityKg
    REAL batteryChargePercent
  PUBLIC:
    CONSTRUCTOR(STRING vin, REAL weight, STRING lidar, STRING fw, REAL payload, REAL battery)
        : AutonomousVehicle(vin, weight, lidar, fw)
      payloadCapacityKg <- payload ; batteryChargePercent <- battery
      DISPLAY "3. DeliveryDrone constructed"
    ENDCONSTRUCTOR
    DESTRUCTOR()
      DISPLAY "DeliveryDrone destroyed"
    ENDDESTRUCTOR
    METHOD DisplayFlightStatus()
      DisplaySpecs()                                // inherited from Vehicle
      DisplayAutonomy()                             // inherited from AutonomousVehicle
      DISPLAY "Payload (kg): ", payloadCapacityKg, "  Battery: ", batteryChargePercent, "%"
    ENDMETHOD
ENDCLASS

BEGIN Main
  DECLARE vPtr AS POINTER TO Vehicle
  DECLARE dPtr AS POINTER TO DeliveryDrone
  vPtr <- NEW DeliveryDrone("VIN-DRN-0042", 12.5, "LDR-77", "v3.2.1", 5.0, 87.0)   // upcast
  vPtr->DisplaySpecs()                              // only Vehicle members are visible
  dPtr <- DOWNCAST vPtr TO POINTER TO DeliveryDrone   // restore full type
  dPtr->DisplayFlightStatus()
  DELETE vPtr                                       // virtual destructor chain
END Main
```

Trace: constructor invocation order from root to leaf, and destruction

Order	Event	Printed
start	NEW DeliveryDrone(...): memory allocated, DeliveryDrone constructor entered	-
1	its initializer list calls AutonomousVehicle(...), whose initializer list calls Vehicle(vin, weight) first	1. Vehicle constructed
2	Vehicle finished, AutonomousVehicle body runs	2. AutonomousVehicle constructed
3	AutonomousVehicle finished, DeliveryDrone body runs	3. DeliveryDrone constructed
4	vPtr->DisplaySpecs() through the base pointer (Vehicle part only)	VIN: VIN-DRN-0042 Base weight (kg): 12.5
5	dPtr->DisplayFlightStatus(): specs, autonomy, drone telemetry	three more lines
6	DELETE vPtr: virtual destructor selects ~DeliveryDrone, then base destructors in reverse	DeliveryDrone destroyed / AutonomousVehicle destroyed / Vehicle destroyed

Question 5: Dynamic Object Pointer Array & Zero-Copy Sorting Engine

Topic: Pointers to Objects, Arrow Operator (->), Pointer Manipulation

Design overview

- **Zero-copy sort:** the array holds only addresses. Comparison dereferences with `catalog[j]->price`. Swapping exchanges the two *pointer variables* (a few bytes), never the Product objects.
- **Ascending bubble sort** with an early-exit flag: if a full pass makes no swap, the array is sorted.
- **Ownership:** each Product is created with NEW, and every pointer is finally released with DELETE (the order after sorting does not matter, because pointers still refer to the original objects).

Structured pseudo-code

```
CLASS Product
  PRIVATE:
    STRING  sku
    REAL    price
    INTEGER stockCount
  PUBLIC:
    CONSTRUCTOR(STRING s, REAL p, INTEGER stock)
      sku <- s ; price <- p ; stockCount <- stock
    ENDCONSTRUCTOR
    METHOD GetPrice() RETURNS REAL
      RETURN price
    ENDMETHOD
    METHOD PrintProduct()
      DISPLAY "SKU: ", sku, "  Price: ", price, "  Stock: ", stockCount
    ENDMETHOD
ENDCLASS

PROCEDURE SortByPrice(REF ARRAY[N] OF POINTER TO Product catalog, INTEGER n)
  INTEGER pass, j
  POINTER TO Product tempPtr
  BOOLEAN swapped
  FOR pass <- 1 TO n - 1 STEP 1
    swapped <- FALSE
    FOR j <- 0 TO n - 1 - pass STEP 1
      IF catalog[j]->GetPrice() > catalog[j+1]->GetPrice() THEN   // compare through pointers
        tempPtr       <- catalog[j]                               // swap POINTERS only
        catalog[j]    <- catalog[j+1]
        catalog[j+1]  <- tempPtr
        swapped <- TRUE
      ENDIF
    ENDFOR
    IF swapped = FALSE THEN
      BREAK                                                       // already sorted
    ENDIF
  ENDFOR
ENDPROCEDURE

BEGIN Main
  DECLARE catalog AS ARRAY[3] OF POINTER TO Product
  INTEGER i
  catalog[0] <- NEW Product("LAPTOP",  1200.0, 15)
  catalog[1] <- NEW Product("MOUSE",     25.0, 200)
  catalog[2] <- NEW Product("MONITOR", 350.0, 40)
  SortByPrice(catalog, 3)
  FOR i <- 0 TO 2 STEP 1
    catalog[i]->PrintProduct()
  ENDFOR
  FOR i <- 0 TO 2 STEP 1
```

```
    DELETE catalog[i]
  ENDFOR
END Main
```

Trace: pointer swaps (objects stay at their original addresses)

Symbolic heap addresses: Laptop at @A, Mouse at @B, Monitor at @C. These never change.

Pass	j	Comparison via ->	Swap pointers?	catalog[0], [1], [2]
start	-	-	-	@A (Laptop 1200), @B (Mouse 25), @C (Monitor 350)
1	0	catalog[0]->price 1200 > catalog[1]->price 25 ? Yes	swap	@B, @A, @C

Pass	j	Comparison via ->	Swap pointers?	catalog[0], [1], [2]
1	1	catalog[1]->price 1200 > catalog[2]->price 350 ? Yes	swap	@B, @C, @A
2	0	catalog[0]->price 25 > catalog[1]->price 350 ? No	no swap	@B, @C, @A
2	-	no swap happened in this pass: swapped = FALSE	BREAK	**final: Mouse 25, Monitor 350, Laptop 1200**

Two pointer exchanges (six pointer assignments) replaced what would have been two full Product copies per swap (three copies each).